

Introduction to CUDA Programming

High Performance Computing

Master in Applied Artificial Intelligence

Nuno Lopes

Parallel Programming on GPU

- ▶ GPUs are devices capable of massive data parallelism through SIMD: single instruction on multiple data
- ▶ When compared to CPU, GPU have more parallel ALU units, focused on arithmetic data operations
 - ▶ To apply the same operation on multiple different data
- ▶ Not suitable for task parallelism, i.e., to run different operations concurrently.

Parallel Programming on GPU

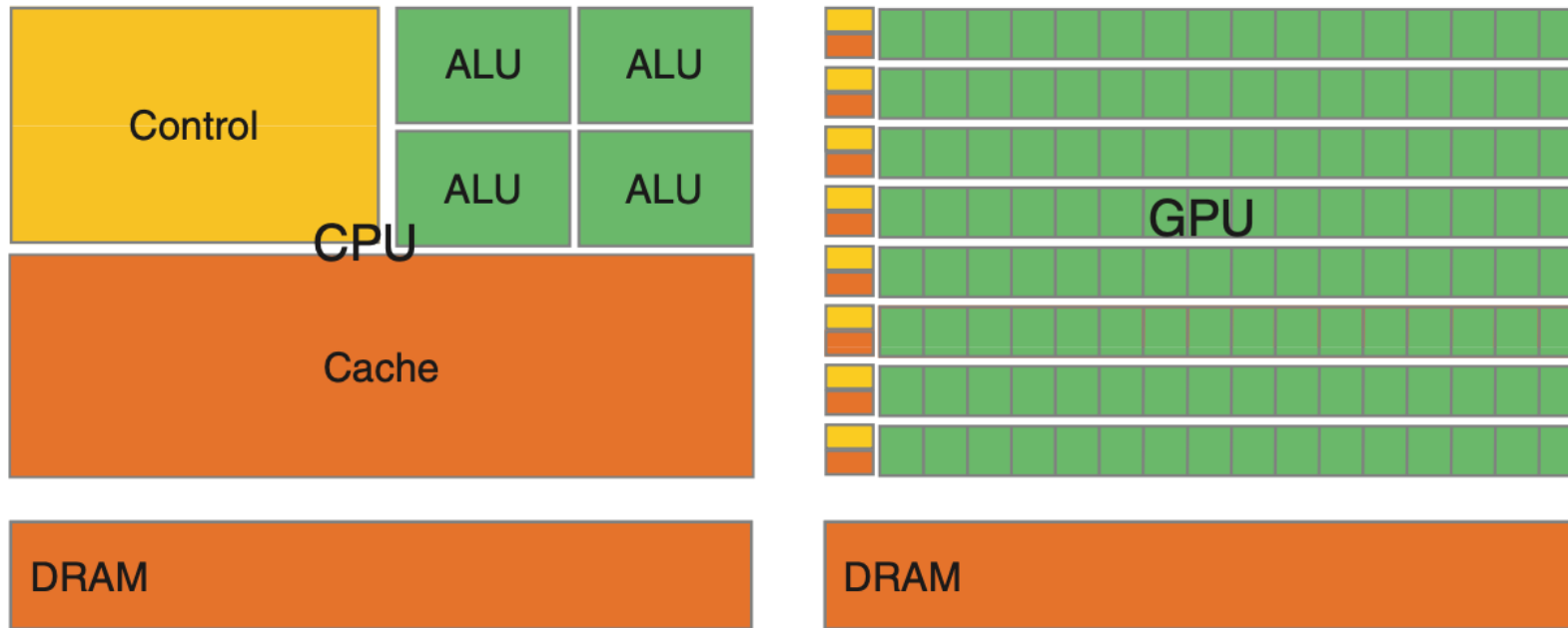


FIGURE 1.1

CPUs and GPUs have fundamentally different design philosophies.

Figures from *Kirk, Hwu, Programming Massively Parallel Processors*

CUDA Programming Architecture

The background of the slide features an abstract design composed of various overlapping triangles and polygons in different shades of blue, ranging from light sky blue to deep navy blue. These geometric shapes are primarily located on the right side of the slide, creating a modern, architectural feel.

CUDA Programming Architecture

- ▶ CUDA program integrates two different types of processors:
 - ▶ *host*: CPU
 - ▶ *device*: GPU
- ▶ Application code includes *host* and *device* codes
 - ▶ Typical C program is CUDA program
 - ▶ Device variables and code is marked with keywords
 - ▶ CUDA annotated program is not valid C program

CUDA Programming Architecture

- ▶ CUDA compiler, Nvidia C Compiler (nvcc) takes cuda (.cu) input program and outputs:
 - ▶ standard C host-only source (for host compiler)
 - ▶ device code (for GPU compiler)
- ▶ The output application runs as usual:
 - ▶ `$ nvcc -o output.exe input.cu`
 - ▶ `$ ./output.exe`

CUDA Kernel and Thread

CUDA Kernel Definition

- ▶ Kernel represents the device function that will be “launched” into the GPU on multiple threads.
 - ▶ Described as standard C/C++ function with `_global_` tag
 - ▶ By default, all functions are `_host_` tagged

	Executed on the:	Only callable from the:
<code><u>_device_</u> float DeviceFunc()</code>	device	device
<code><u>_global_</u> void KernelFunc()</code>	device	host
<code><u>_host_</u> float HostFunc()</code>	host	host

FIGURE 3.12

CUDA C keywords for function declaration.

CUDA Kernel Definition

- ▶ Kernel function is specified as C code
- ▶ Returns void, parameters serve as input and output
- ▶ Local variables are independent for each thread.

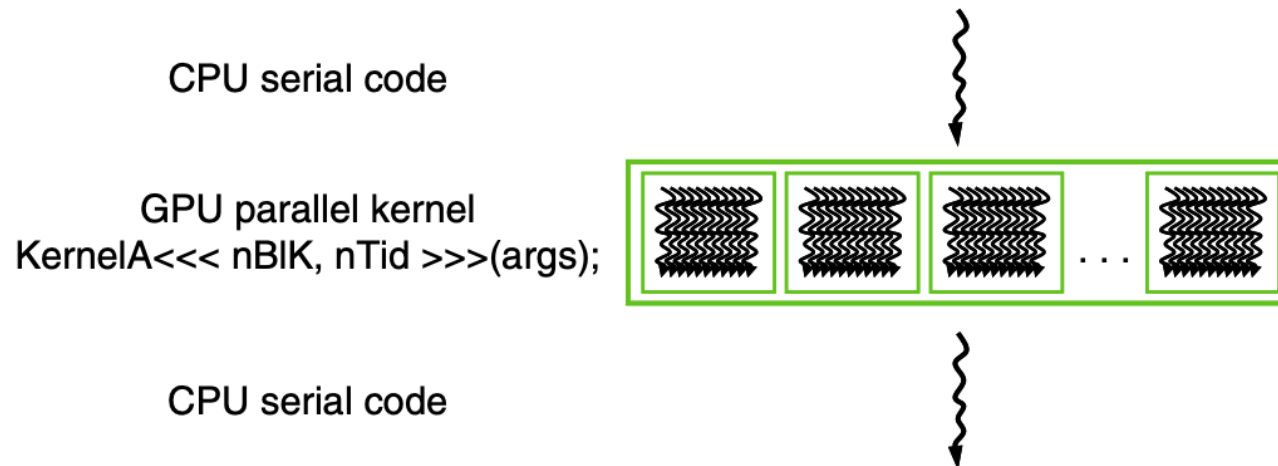
```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__
void vecAddKernel(float* A, float* B, float* C, int n)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if(i<n) C[i] = A[i] + B[i];
}
```

FIGURE 3.11

A vector addition kernel function and its launch statement.

Kernel Launch

- ▶ The execution of a kernel function on the device is named “kernel launch”.
 - ▶ It is the host that starts the launch;
 - ▶ The kernel specifies the code for a single thread;
 - ▶ Launching a kernel specifies the number of threads to be used.



Thread Hierarchy

- ▶ Threads are organised in a double hierarchy: grid & block.
- ▶ When launching a kernel, a grid is created.
- ▶ A grid can have a number of blocks of threads (in up to 3 dimensions).
- ▶ A block can have a number of threads (in up to 3 dimensions).
- ▶ Example for launching grid with 4 blocks, each with 32 threads:
 - ▶ `KernelFunc <<< 4, 32 >>> (parameters);`

Thread Hierarchy

- ▶ Example:
- ▶ Grid of 3 x 2 (6) blocks,
- ▶ each with 4 x 3 (12) threads,
- ▶ totalling 72 threads.

...

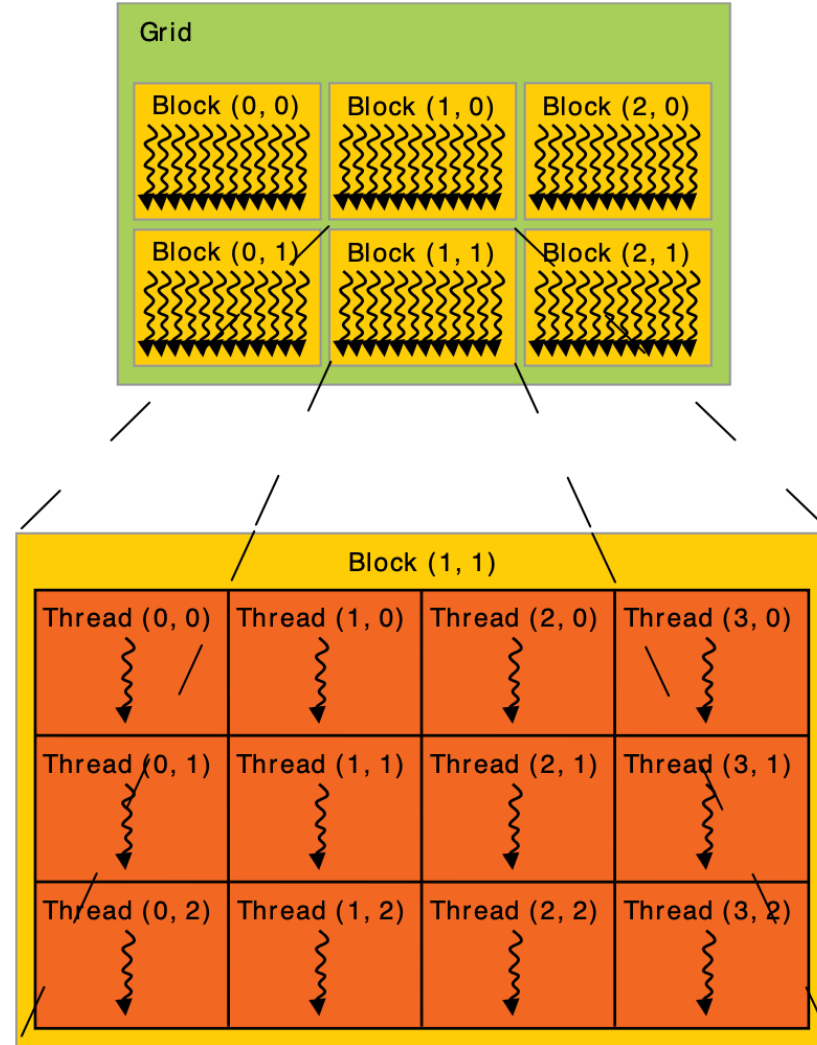
```
dim3 blocksPerGrid( 3, 2);  
dim3 threadsPerBlock( 4, 3);
```

```
KernelFunc <<<blocksPerGrid, threadsPerBlock>>> (params);
```

...

Source: Nvidia, Cuda C++ Programming Guide, [web link](#), [v11.6 pdf](#)

Grid of Thread Blocks



Thread Identification

Each thread has available the following (private) variables:

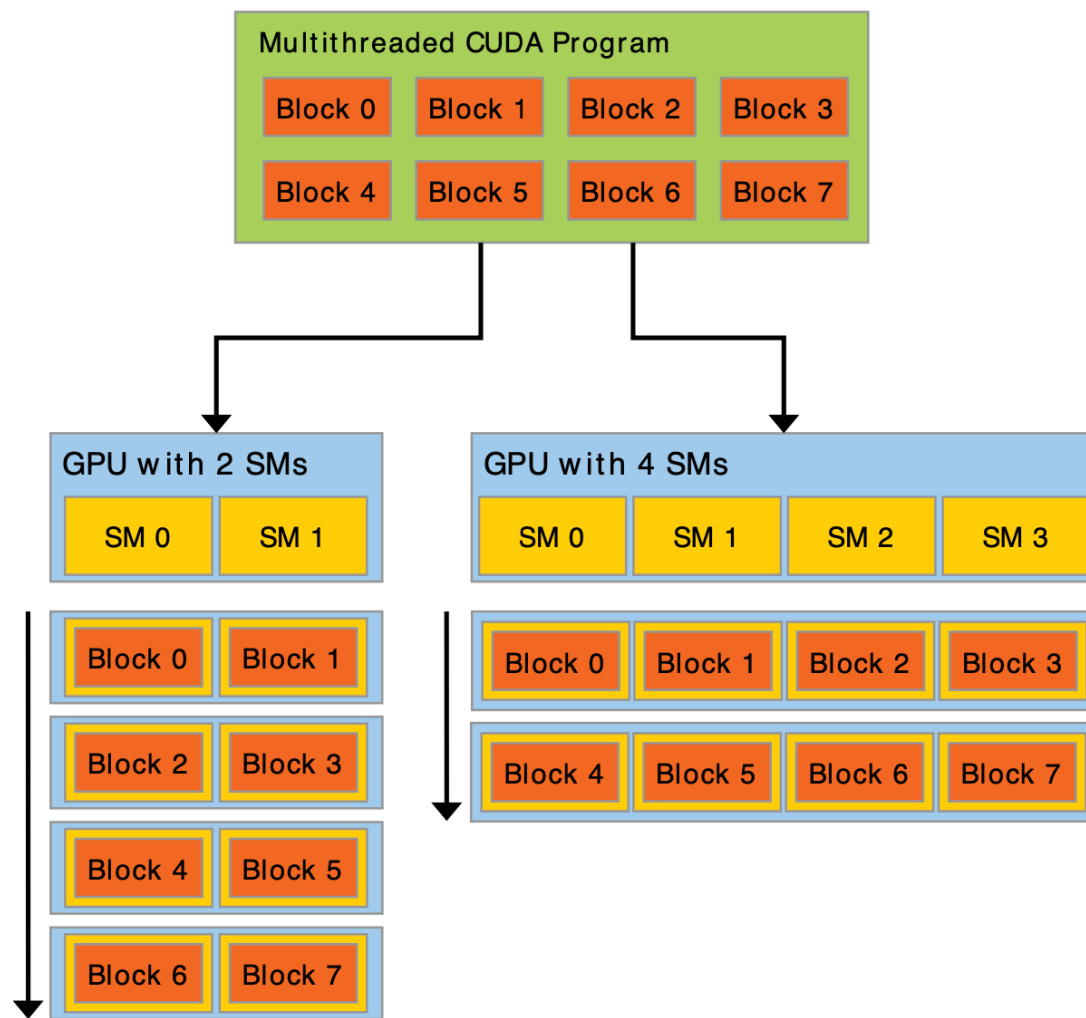
- ▶ **gridDim**: grid dimension (x, y, z coordinates)
- ▶ **blockDim**: block dimension (x, y, z coordinates)
- ▶ **blockIdx**: block identification (per thread)
- ▶ **threadIdx**: thread identification

Block-oriented Kernel Scheduling

- ▶ The scheduling of kernel within a grid is block-oriented.
- ▶ Blocks are required to run independent from each other, so that device can schedule them in any order.
- ▶ Depending on the # hardware resources, Streaming Multiprocessors (SM), it is possible to run more than one block at a time.
- ▶ SM schedules a group of threads of a block in parallel, called a **warp**.
- ▶ Warps have size of multiple of 32 threads, depending on hardware.

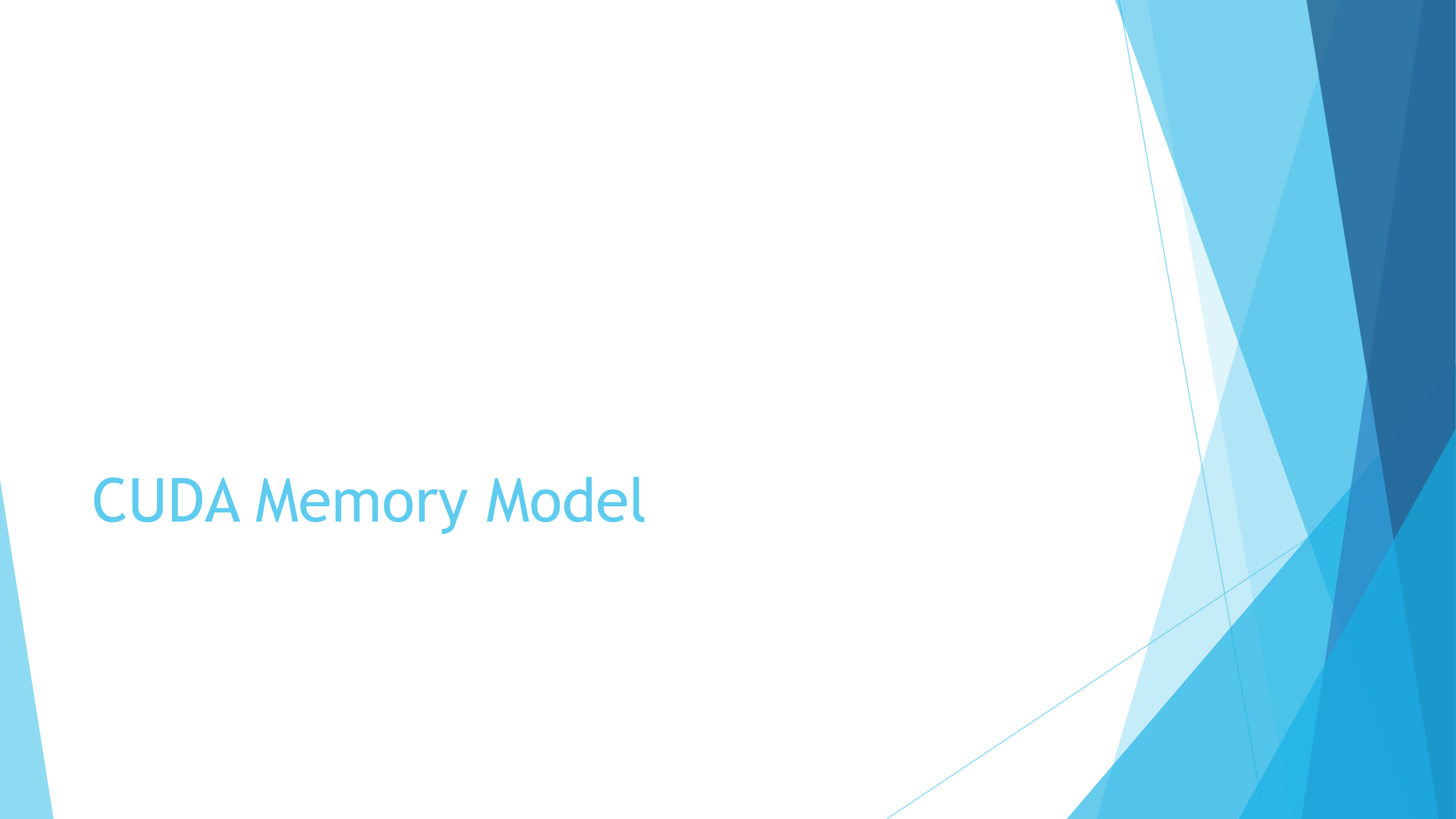
Block-oriented Kernel Scheduling

Figure 3. Automatic Scalability



Source: Nvidia, Cuda C++ Programming Guide

CUDA Memory Model

The background of the slide features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side and bottom of the frame, creating a modern, tech-oriented aesthetic.

Memory Model CPU-GPU

- ▶ Host has its own RAM: *host memory*
- ▶ Device also has its own RAM: *global memory*
- ▶ It is necessary to transfer data between these memories:
 - ▶ initially from host to device (with input data)
 - ▶ in the end, from the device to the host (with the result)

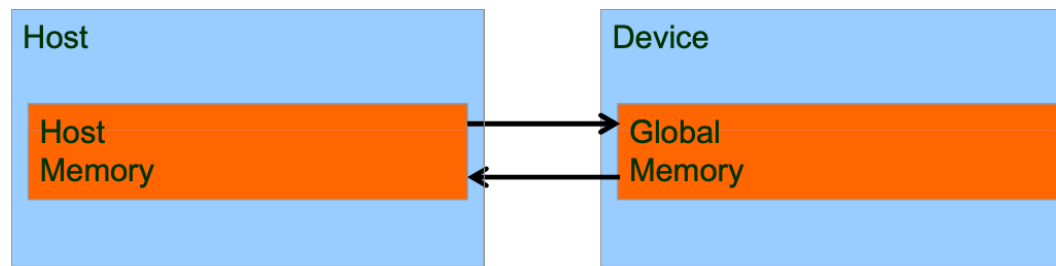


FIGURE 3.6

Host memory and device global memory.

Device Memory Initialisation & Transfer

- ▶ Memory in the device Global Memory, needs to be allocated, from the host.
 - ▶ `cudaMalloc( address of pointer, size)`
 - ▶ `cudaFree( pointer )`
- ▶ An additional transfer function is available:
 - ▶ `cudaMemcpy( destination, source, bytes, direction )`
 - ▶ Direction:
 - ▶ `cudaMemcpyHostToDevice,`
 - ▶ `cudaMemcpyDeviceToHost`

Cuda Device Memory Layout

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

Host code can

- Transfer data to/from per grid **global** and **constant** memories

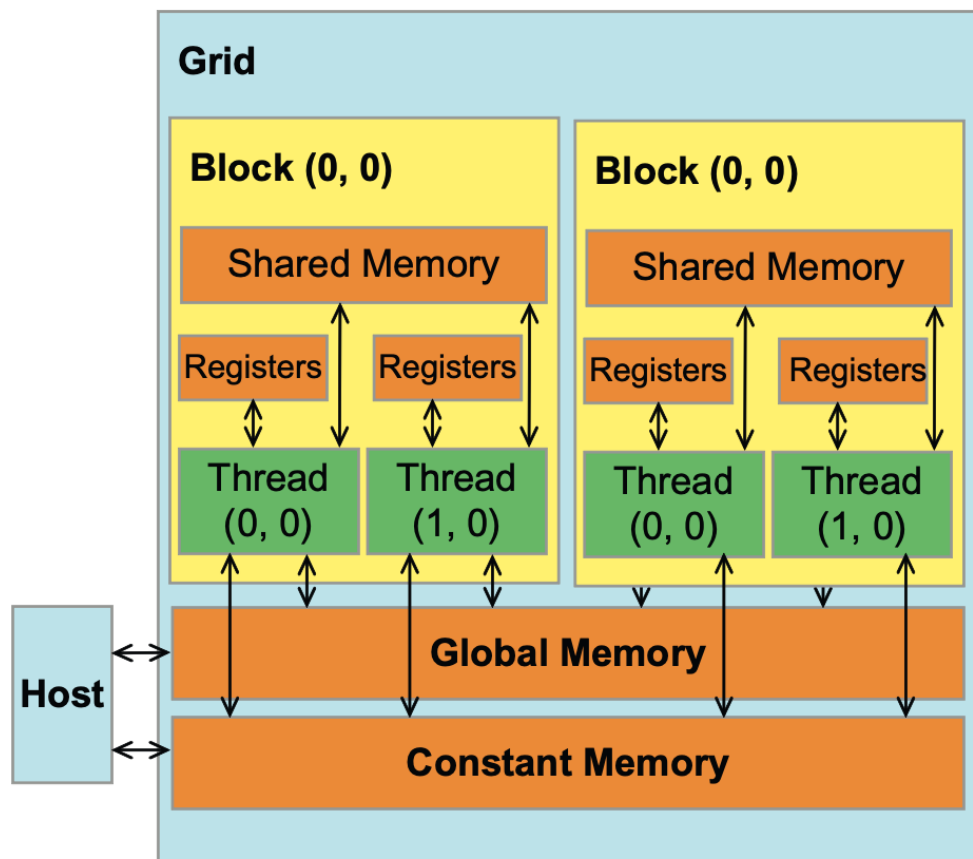


FIGURE 5.2

Overview of the CUDA device memory model.

Cuda Device Memory Layout

- ▶ The specification of the storage type, scope and lifetime of device variables is defined by tag.
 - ▶ Automatic scalar (non array) vars are register based.
 - ▶ Local thread memory is physically on Global Memory.

Table 5.1 CUDA Variable Type Qualifiers

Variable Declaration	Memory	Scope	Lifetime
Automatic variables other than arrays	Register	Thread	Kernel
Automatic array variables	Local	Thread	Kernel
<code>__device__ __shared__ int SharedVar;</code>	Shared	Block	Kernel
<code>__device__ int GlobalVar;</code>	Global	Grid	Application
<code>__device__ __constant__ int ConstVar;</code>	Constant	Grid	Application

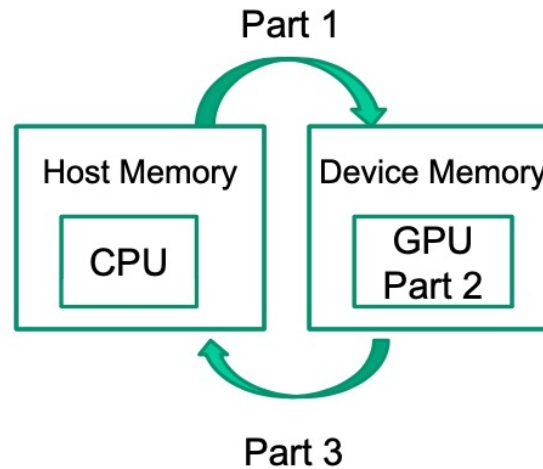
CUDA application example

Structure of Vector Addition program

```
#include <cuda.h>
...
void vecAdd(float* A, float*B, float* C, int n)
{
    int size = n* sizeof(float);
    float *A_d, *B_d, *C_d;
    ...
    1. // Allocate device memory for A, B, and C
       // copy A and B to device memory

    2. // Kernel launch code – to have the device
       // to perform the actual vector addition

    3. // copy C from the device memory
       // Free device vectors
}
```



Structure of Vector Addition program

```
void vecAdd(float* A, float* B, float* C, int n)
{
    int size = n * sizeof(float);
    float *d_A, *d_B, *d_C;

    cudaMalloc((void **) &d_A, size);
    cudaMemcpy(d_A, A, size, cudaMemcpyHostToDevice);
    cudaMalloc((void **) &d_B, size);
    cudaMemcpy(d_B, B, size, cudaMemcpyHostToDevice);

    cudaMalloc((void **) &d_C, size);
    vecAddKernel<<ceil(n/2560), 256>>>(d_A, d_B, d_C, n);

    cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost);
    // Free device memory for A, B, C
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

FIGURE 3.14

A complete version of `vecAdd()`.

Structure of Vector Addition program

```
// Compute vector sum C = A+B
// Each thread performs one pair-wise addition
__global__
void vecAddKernel(float* A, float* B, float* C, int n)
{
    int i = threadIdx.x + blockDim.x * blockIdx.x;
    if(i<n) C[i] = A[i] + B[i];
}
```